PROJECT DIAGNOSTIC & COMPLIANCE REPORT

Author: Gnaneswar Dowluri
Role: AIML Engineering Intern
Date: July 22, 2026
Project: Single-Document FAQ RAG Pipeline Challenge

---

1. Executive Summary

This technical report details the diagnostic investigations, code-level resolutions, and premium frontend/backend upgrades implemented on the prototype single-document Retrieval-Augmented Generation (RAG) FAQ Bot. The prototype application was delivered in a completely broken state, characterized by text indexation gaps, inverted similarity retrievals, and stripped context prompt bodies. These defects resulted in total system failure, leading to unit test crashes and hallucinated or empty LLM answers.

By performing a systematic code audit, all three critical RAG pipeline bugs were successfully resolved in the workspace. In addition to resolving the core bugs, the pipeline's format support was expanded to parse Word Documents (.docx) and Markdown (.md) files. A high-fidelity, dark-themed Streamlit user interface was engineered, and the test suite was expanded from 8 to 13 tests. This report details the implementation mechanics and verifies that all tests currently achieve 100% compliance.

2. RAG Pipeline Architecture

The FAQ Bot operates on an industry-standard RAG architecture, which grounds Large Language Model queries using dynamic text search over indexed documents. The pipeline operates as follows:

- **Document Extraction:** Parses user-submitted files (.pdf, .txt, .docx, .md) into raw clean strings.
- **Text Chunking:** Splits the document text into smaller, overlapping windows to maintain local semantic continuity and satisfy LLM prompt token limits.
- **Vector Indexing:** Generates normalized vector representations (embeddings) for all text chunks via scikit-learn's TfidfVectorizer or SentenceTransformers.
- **Cosine Similarity Retrieval:** Compares query vector embeddings against indexed chunks and retrieves the top-k matches above a similarity score threshold.
- **Grounded LLM Prompting:** Injects the relevant context into the LLM system prompt template to force precise, grounded answers.

3. Detailed Pipeline Diagnostics & Bug Fixes

Below are the three core engineering bugs identified in the original zip archive, alongside their symptoms, impacts, and code resolutions.

Bug #1: Sliding Window Step Overlap (Text Chunking)

Symptom: The chunking module created gaps between text boundaries, missing entire paragraphs during indexing.

Core Flaw: In *src/chunk.py*, the sliding window step was calculated as *step = chunk_size + overlap* instead of subtracting the overlap to shift the window back.

Code Diff Comparison:

```
# Buggy Baseline (Zip File):
step = chunk_size + overlap

# Corrected Code (Workspace File):
step = max(1, chunk_size - overlap)
```

Bug #2: Cosine Similarity Sort Inversion (Retrieval Indexing)

Symptom: The search retrieved the least matching chunks instead of highly relevant context.

Core Flaw: In *src/retrieve.py*, *np.argsort(scores)* sorted scores in ascending order. Slicing the first elements retrieved the lowest scoring matches.

Code Diff Comparison:

```
# Buggy Baseline (Zip File):
ranked_indices = np.argsort(scores)
top_indices = ranked_indices[:min(top_k, len(chunks))]

# Corrected Code (Workspace File):
ranked_indices = np.argsort(scores)[::-1] # Reverse for descending order
top_indices = ranked_indices[:min(top_k, len(chunks))]
```

Bug #3: Stripped Prompt Context (LLM Generation)

Symptom: The offline/live LLM generator reported it could not find answers despite relevant text.

Core Flaw: In *src/generate.py*, the prompt was formatted with an empty string literal *context=''*, neglecting the compiled context block.

Code Diff Comparison:

```
# Buggy Baseline (Zip File):
prompt = PROMPT_TEMPLATE.format(context="", query=query)

# Corrected Code (Workspace File):
prompt = PROMPT_TEMPLATE.format(context=context_str, query=query)
```

4. Workspace Enhancements & Premium Refactoring

To deliver an enterprise-grade utility, the codebase was updated with three additional features:

4.1 Extended File Support (DOCX & MD Parsing)

The text extractor in *src/extract.py* was refactored to parse Microsoft Word Document (.docx) files (including paragraphs and structured table rows) and Markdown (.md) raw text layouts. Unit tests were added in *tests/test_extract.py* to validate extraction integrity.

4.2 Premium Dark-Glassmorphic User Interface

The Streamlit frontend (*app.py*) was completely redesigned to create a premium visual experience:

- Injected the modern **Outfit** sans-serif typography via Google Fonts.
- Styled layout containers using glassmorphic borders and custom dark background cards.
- Designed color-coded live API status badges in the sidebar panel.
- Added a **Workspace Inspector** tab allowing real-time audit of chunking parameters, character distributions, and segmented document texts.

4.3 Expanded Testing Framework

The testing framework was expanded from 8 unit tests to 13 comprehensive unit tests, verifying DOCX parsing, markdown styling, and generator prompt assembly. All tests pass with a 100% success rate.

5. Verification & Test Metrics

The system was validated using the standard Python test discover runner, yielding:

Test File	Tests Ran	Passing Status	Target Module Verified
test_chunk.py	3	PASS	src.chunk (Chunking & Overlap)
test_retrieve.py	2	PASS	src.retrieve (Similarity & Sort)
test_generate.py	3	PASS	src.generate (Prompt Generation)
test_extract.py	5	PASS	src.extract (TXT/PDF/DOCX/MD)

Report Certified By:
Gnaneswar Dowluri
AIML Engineering Intern